

```

1  ( *
2
3          CS51 Lab D
4          Improving Debugging Skills
5  ( *
6
7          SOLUTION
8
9  (*=====
10 Readings:
11
12     This lab has no prerequisite textbook reading.
13
14 Objective: In this lab, you'll improve your debugging skills by
15 applying fundamental debugging ideas to improving code.
16
17     In the exercises that follow, some functions may have bugs, so that
18     their behavior may not match the intended behavior described in the
19     comments. Your job is to find and fix all of the bugs.
20 =====*)
21
22 (*=====
23 Part 0: Important aspects of debugging
24
25 You may not have thought explicitly about the debugging process, but
26 doing so can provide you with valuable skills in the process. Here are
27 some of the major aspects of the debugging process.
28
29     Identification
30
31         Read error messages in detail. They often provide not just the
32         nature of the error, but an approximate location.
33
34         Set up unit tests for individual functions. Unit tests can
35         identify bugs in your code by finding cases that don't match
36         the behavior you intended. Try to specify unit test cases that
37         cover all of the important paths through the code. A good
38         technique is to put the unit tests in a separate file that
39         references the file with the functions to be tested. Then,
40         whenever you make changes to the functions, you can rerun the
41         test file to make sure that you haven't introduced bugs in
42         previously working code.
43
44     Localization
45
46         When you first identify a bug, you may not know where in the
47         code base the bug actually lives. You'll need to localize the
48         bug -- finding its location in the code base.
49
50         In tracking down problems in larger codebases, eliminate
51         portions of the code to generate the minimal codebase that
52         demonstrates the problem. Breaking the code into smaller parts

```

can allow localization to one of the parts, as they can be unit-tested separately.

Simplification

When confronted with an error exhibited on a large instance, try to simplify it to find the minimal example that exhibits the problem.

Reproduction

Try alternate examples to see which ones exhibit the problem. The commonalities among the examples that exhibit the problem can give clues as to the problem.

Diagnosis

Verify that invariants that should hold in the code actually do, with assertions or other constructs. (The `Absbook.verify` function can be especially useful in verifying invariants of the arguments and return value.) Conduct experiments to test your theory of what has gone wrong.

Correction

Generate git commits to save a version of the code so that you can confidently make changes to the code while you are experimenting, knowing that you'll be able to return to earlier versions.

Maintenance

Code that was once working can become buggy as changes are made either to the code itself or to code that it uses. It's thus helpful to retest code when changes are made to it or its environment. Fortunately, unit test files are ideal for this process. Rerunning the unit tests liberally allows us to verify that working code hasn't regressed to a buggy state. (The process is referred to in the literature as "regression testing" for this reason. See https://en.wikipedia.org/wiki/Regression_testing.)

*)

(=====

Part 1: Some finger exercises in debugging

In this part, we'll provide implementations of a few simple functions. These functions may or may not have bugs in them. You'll proceed through five steps:

1. Read the function definition, including the top-level comment, to give you an idea of what the function is intended to do.

```

107 2. Write a full set of unit tests for each of the functions in the
108     file `part1_tests.ml`. The comments introducing each function may
109     give information about the intended behavior of each function.
110     *** Do not skip this step. ***
111
112 3. Once you've written all of the unit tests, compile and run the
113     unit tests
114
115     % ocamlbuild -use-ocamlfind part1_tests.byte
116     % ./part1_tests.byte
117
118 4. For each of the functions, find a value for the function's
119     argument on which the function fails, if there is one. (If
120     you've built your unit tests well, they should uncover such a
121     value directly.) Record the failing input by let-defining the
122     corresponding `_fails_on` value to be `Some v` where `v` is the
123     failing input if there is one, or `None` if the function is not
124     buggy.
125
126 5. Revise each function to eliminate any bugs that you found. While
127     you're at it, you should probably deal with any warnings that
128     arise as well.
129
130 We've done the first of these exercises, the `abs` function, for you
131 to give you the idea.

```

132 1. ORIGINAL VERSION

```

134
135 (* abs x -- Returns the absolute value of the integer `x` *)
136 let abs x =
137     if x < ~-1 then ~- x else x
138

```

139 2. UNIT TESTS (These would be added to `part1\_tests.ml`.)

```

140
141 unit_test (abs 0 = 0) "abs zero";
142 unit_test (abs 1 = 1) "abs one";
143 unit_test (abs max_int = max_int) "abs maxint";
144 unit_test (abs min_int = min_int) "abs minint";
145 unit_test (abs (-0) = 0) "abs zero";
146 unit_test (abs (-1) = 1) "abs neg one";
147 unit_test (abs (-max_int) = max_int) "abs neg maxint";
148 unit_test (abs (-min_int) = min_int) "abs neg minint";
149

```

150 3. COMPILE AND RUN

```

151
152 % ocamlbuild -use-ocamlfind part1_tests.byte
153 % ./part1_tests.byte
154 ...
155 abs zero passed
156 abs one passed
157 abs maxint passed
158 abs minint passed
159 abs zero passed
160 abs neg one FAILED          <-- This'll be helpful!

```

```

161     abs neg maxint passed
162     abs neg minint passed
163     ...
164     - : unit = ()
165
166     4. FAILING INPUT
167
168     let abs_fails_on = Some (-1)
169
170     5. REVISED VERSION
171
172     (* abs x -- Returns the absolute value of the integer `x` *)
173     let abs x =
174         if x < 0 then ~- x else x
175     *)
176
177     (* ..... *)
178     (* last_element lst -- Returns the last element of `lst` as an option;
179        `None` if there is no last element *)
180     let rec last_element lst =
181         match lst with
182         | [] -> None
183         | [x] -> Some x
184         | _ :: tail -> last_element tail
185
186     let last_element_fails_on = Some [1; 2]
187
188     (* ..... *)
189     (* sum_to_n n -- Returns the sum of integers from 1 to `n` *)
190     let rec sum_to_n n =
191         if n = 0 then n
192         else n + sum_to_n (n - 1)
193
194     let sum_to_n_fails_on = None
195
196     (* ..... *)
197     let describe_list lst =
198         match lst with
199         | [] -> "Empty list"
200         | [_x] -> "Singleton list"
201         | _ :: _ -> "Multiple list"
202
203     let describe_list_fails_on = Some [1]
204
205     (* ..... *)
206     (* one_empty lst -- Returns `true` if and only if `lst` is a list
207        containing *exactly* one element, where that element is itself the
208        empty list. (For example, `one_empty [[]] = true` but
209        `one_empty [] = false` and `one_empty [[1]] = false`.) *)
210     (* The bug is that the inner `match x with ...` is unparenthesized, so
211        OCaml treats the trailing `| _ :: _ -> false` clause as another case
212        of the *inner* match (on `x`) rather than of the outer match (on
213        `lst`). This leaves the outer match with no clause for lists of
214        length two or greater (the compiler warns that the match is

```

```

215     non-exhaustive), so `one_empty [[]; [1]]` and similar inputs raise
216     `Match_failure` instead of returning `false`.
217
218     One fix is simply to bracket the inner match with `begin ... end`
219     (or with parentheses) so that its scope is clear. *)
220 let one_empty lst =
221     match lst with
222     | [] -> false
223     | [x] ->
224         begin
225             match x with
226             | [] -> true
227             | _ -> false
228         end
229     | _ :: _ -> false
230
231 (* Another fix is to avoid the nested match entirely, using a single
232    match with deeper patterns. This version is arguably clearer.
233    let one_empty lst =
234        match lst with
235        | [[]] -> true
236        | _ -> false
237    *)
238
239 let one_empty_fails_on = Some [[]; [1]]
240
241 (=====
242 Part 2: Debugging set operations
243
244 In this part, you will apply your debugging skills to debugging an
245 implementation of set operations (union, intersection, etc.).
246
247 *****
248 In this lab, sets of integers will be represented as `int list`s
249 whose elements are in *sorted order* with *no duplicates*. All
250 functions can assume this invariant and should deliver results
251 satisfying it as well.
252 *****
253
254 To get you started on debugging, we've placed a few unit tests for
255 some of the functions in the file `part2_tests.ml`. Compile and run
256 these tests to see how the functions are working so far.
257
258 % ocamlbuild -use-ocamlfind part2_tests.byte
259 % ./part2_tests.byte
260
261 What do you notice? Does this give you an idea on where to start
262 debugging?
263
264 =====
265 Part 2A: Some utilities for checking the sorting and no-duplicates
266 conditions.
267 *)
268

```

```

269 (* is_sorted lst -- Returns `true` if and only if `lst` is a sorted
270    list *)
271 let is_sorted (lst : 'a list) : bool =
272   lst = List.sort Stdlib.compare lst ;;
273
274 (* dups_sorted lst -- Returns the number of duplicate elements in
275    `lst`, a sorted list of integers. For example
276
277       # dups_sorted [1;2;5;5;5;5;5;5;6;7;7;9] ;;
278       - : int = 6
279    *)
280 let rec dups_sorted (lst : 'a list) : int =
281   match lst with
282   | [] -> 0
283   | [_] -> 0
284   | first :: (second :: _rest as second_rest) ->
285     dups_sorted second_rest
286     + if first = second then 1 else 0 ;;
287
288 (* SOLUTION: The first of the two failing unit tests is
289
290     dups_sorted [1; 3; 4; 4; 6; 12; 13; 13; 15]
291
292     which should be 2 (the duplicate 4 and 13). Running it directly, we get
293
294     # dups_sorted [1; 3; 4; 4; 6; 12; 13; 13; 15] ;;
295     - : int = 1
296
297     We can start by dropping all of the unduplicated elements to try to find a
298     shorter failing example. Sure enough,
299
300     # dups_sorted [4; 4; 13; 13] ;;
301     - : int = 1
302
303     In fact, it looks like any matches after the first one is ignored.
304
305     # dups_sorted [4; 4; 4; 13; 13; 15; 15; 15; 72; 72] ;;
306     - : int = 1
307
308     First off, we'll definitely want to add some tests like these to
309     `part2_tests.ml` to aid our testing down the road.
310
311     Returning to our simplest case,
312
313     dups_sorted [4; 4; 13; 13] ;;
314
315     the first match will happen based on the final match case,
316
317     | first :: second :: rest ->
318       if first = second then 1 else 0
319       + dups_sorted rest
320
321     which is *attempting* to add 1 or 0 to the recursive count of
322     duplicates. Unfortunately, and despite the intention expressed by

```

the indentation, what is actually getting calculated is better indicated by this layout:

```
| first :: second :: rest ->
  if first = second
  then 1
  else 0 + dups_sorted rest
```

that is, in the case of a duplicate, the recursive call is never even made! This bug would therefore show up in any example that has more than one duplicate. This is easily repairable, either using parentheses to override the precedence or by swapping the two terms being added, like this:

```
| first :: second :: rest ->
  dups_sorted rest
  + if first = second then 1 else 0
```

Making this change repairs the bug:

```
# dups_sorted [4; 4; 13; 13] ;;
- : int = 2
# dups_sorted [1; 3; 4; 4; 6; 12; 13; 13; 15];;
- : int = 2
```

But that's not sufficient. The test case `dup in end` is still failing. We can check it at the command line.

```
# dups_sorted [1; 3; 4; 6; 10; 12; 13; 15; 15] ;;
- : int = 0
```

Can we find a simpler case that fails. We'll start by dropping elements from the list one by one:

```
# dups_sorted [1; 3; 4; 6; 10; 12; 13; 15; 15] ;;
- : int = 0
# dups_sorted [3; 4; 6; 10; 12; 13; 15; 15] ;;
- : int = 1
# dups_sorted [4; 6; 10; 12; 13; 15; 15] ;;
- : int = 0
# dups_sorted [6; 10; 12; 13; 15; 15] ;;
- : int = 1
```

It looks like the duplicate is found only when it's at an even numbered index. The simplest instance, then should be (sure enough)

```
# dups_sorted [13; 15; 15] ;;
- : int = 0
```

That's a short enough example that we can simply play computer and see what the code would do. In pattern matching this list, `first` would be `13`, `second` would be `15`, and `rest` would be the list `[15]`. Since 13 and 15 are different, we'll add 0 to the recursive call on the list `[15]`. But that list has no duplicates, so the

```

377     sum would be `0`.
378
379     By now, you've probably seen the bug. The recursive call applies
380     to the tail of `lst` after the first two elements (`first` and
381     `second`). What if the second and third elements are duplicates?
382     The tail won't find that pair! We need to hang on to the tail after
383     just the first element, so as to recur on it. This can be done in a
384     variety of ways. One is to embed a second match:
385
386     | first :: rest ->
387         match rest with
388         | [] -> failwith "dups_sorted: can't happen"
389         | second :: _ ->
390             dups_sorted rest
391             + if first = second then 1 else 0
392
393     A much cleaner solution uses the `as` construct in patterns to
394     allow the naming of subparts of a pattern match:
395
396     | first :: (second :: _rest as second_rest) ->
397         dups_sorted second_rest
398         + if first = second then 1 else 0 ;;
399 *)
400
401 (* is_set lst -- Returns `true` if and only if lst represents a set,
402    with no duplicates and elements in sorted order. *)
403 let is_set (lst : 'a list) : bool =
404     is_sorted lst && dups_sorted lst = 0 ;;
405
406 (*=====
407 Part 2B: Set operations -- member, union, and intersection
408
409 Below we provide code for computing membership, intersections, and
410 unions of sets represented by lists with the stated invariant.
411
412 Check out the unit tests for these in `part2_tests.ml`. Augment the
413 tests until you're satisfied that you've fully tested these functions,
414 making any needed changes as you go.
415
416 We'll test them further on larger examples in the next part, Part
417 2C. *)
418
419 (* member elt set -- Returns `true` if and only if `elt` is an element
420    of `set` (represented as above). Search can stop early based on
421    sortedness of `set`. *)
422 let rec member elt set =
423     match set with
424     | [] -> false
425     | hd :: tl ->
426         if elt = hd then true
427         else if elt < hd then false
428         else member elt tl ;;
429
430 (* SOLUTION: We've added a good set of unit tests for the `member`

```

```

431     function. These show that the function seems to be working
432     correctly. *)
433
434
435 (* SOLUTION: Check out the tests we added to the testing file. With no changes
436    in the definition of `union`, some of these tests, even simple
437    ones, fail. For instance,
438
439        # union [1] [1] = [1] ;;
440        - : bool = false
441
442    What happens in the code when the two head elements are the
443    same? We take the head element of the first set, together with
444    remaining elements *including* the redundant head element of the
445    second set. This leads to two copies of the head element, violating
446    the no-duplicates invariant. Instead, we need only keep one of the
447    identical head elements. (While we're at it, we clean up the base
448    case pattern matches as well.) *)
449
450 (* union set1 set2 -- Returns a list representing the union of the
451    sets `set1` and `set2` *)
452 let rec union s1 s2 =
453   match s1, s2 with
454   | [], _ -> s2
455   | _, [] -> s1
456   | hd1 :: tl1, hd2 :: tl2 ->
457     if hd1 = hd2 then
458       hd1 :: union tl1 tl2
459     else if hd1 < hd2 then
460       hd1 :: union tl1 s2
461     else (* hd1 > hd2 *)
462       hd2 :: union tl2 s1 ;;
463
464 (* intersection set1 set2 -- Returns a list representing the
465    intersection of the sets `set1` and `set2` *)
466 let rec intersection s1 s2 =
467   match s1, s2 with
468   | [], _ -> []
469   | _, [] -> []
470   | hd1 :: tl1, hd2 :: tl2 ->
471     if hd1 = hd2 then hd1 :: intersection tl1 tl2
472     else if hd1 < hd2 then intersection tl1 s2
473     else intersection s1 tl2 ;;
474
475 (* SOLUTION: Even a small set of tests, as we've added in the testing file,
476    finds major problems with this implementation of `intersection`. Once
477    the failures are noticed, the bug -- incorrect parity of when to end
478    the search -- is clear and easily repaired.
479    *)
480
481 (*=====
482 Part 2C: Scaling up the testing
483
484 The file `labD_examples` contains a couple of larger examples of sets

```

```

485 represented as lists (`example1` and `example2`). The `part2_tests.ml`
486 file contains a few tests based on these larger examples, which are
487 commented out at the moment. Uncomment them now and rerun the unit
488 tests. What do you notice?
489
490 More bugs to debug. Where do you think the problems lie? Remaining
491 bugs in the functions above? In the examples? In the tests themselves?
492
493 You're on your own to figure out what's going on and correct the
494 problems, wherever they might be.
495 *)
496
497 (* SOLUTION: Adding in the tests on the larger examples reveals some
498 failures. This is perhaps surprising if you've heavily tested and
499 debugged the functions involved. What could be going on? There are
500 three possibilities.
501
502 1. There are remaining bugs in the functions above.
503 2. The unit tests are incorrect. (They check for the wrong values
504 of the test expression.)
505 3. The values used in the unit tests are problematic.
506
507 Let's treat them serially.
508
509 1. We've added a bunch of unit tests that seem to cover all of the
510 cases. Let's assume the functions are okay for now, but may need
511 to return to that assumption.
512
513 2. We can manually double check the unit tests.
514
515     Is 284 actually a member of `example1`? Yes, it is, as
516     verified by a simple search in the file.
517
518     Is 284 actually a member of `example2`? Ditto.
519
520     Is 284 thus a member of the intersection of the two examples?
521     It should be.
522
523     Similarly for the union.
524
525     A similar process reveals that the other tests are correctly
526     stated. Hmm.
527
528 3. Eyeballing the values used in the tests -- `example1` and
529 `example2` -- they seem to look fine. No duplicates or out of
530 order items at first glance. But eyeballing such a large data
531 structure isn't ideal. Computers can do the checking better than
532 humans. In fact, we have a function to check for us. Let's add
533 unit tests to `part2_tests.ml` that verify that the two examples
534 are in fact invariant-obeying sets:
535
536     # is_set example1 ;;
537     - : bool = true
538     # is_set example2 ;;

```

539 - : bool = false

540

541 The latter unit test fails!

542

543 The lesson here is that sometimes what needs debugging is **the*
544 unit tests themselves*. Let's debug the problem with ``example2``.

545

546 Does it fail the test because it has duplicates or because it
547 isn't sorted or both?

548

549 # is_sorted example2 ;;

550 - : bool = false

551 # dups_sorted example2 ;;

552 - : int = 0

553

554 Apparently, the problem is in the sorting.

555

556 How can we find the exact location of the bug in ``example2``?
557 More careful eyeballing is too painful. Again, it's better to
558 let the computer do the work.

559

560 One good and very general approach to localizing a problem in a
561 large program is to **use binary search**. We can separately test
562 the first half and the second half of the code to see which half
563 the problem occurs in. We do the same in the offending half to
564 find the bad quarter. Eventually, we'll have narrowed the search
565 down to a short enough piece of code that eyeballing may be
566 sufficient. (The process shouldn't require many divisions
567 because of the tremendous abbreviating power of logarithms.)

568

569 Another approach is to **use computer power** to aid in the
570 search. Rather than our finding the location of the problem, we
571 write code to do it for us. For instance, we can compare
572 ``example2`` with its sorted version, and find the first place
573 where they differ. That should give us a clue as to what's going
574 on.

575

576 # List.find (fun (x, y) -> x <> y)

577 (List.combine example2

578 (List.sort Stdlib.compare example2)) ;;

579 - : int * int = (1872, 1076)

580

581 The problem seems to be that either 1872 or 1076 is out of
582 order. Now, reexamining the definition of ``example2`` verifies that
583 1872 is way early.

584

585 Now we turn to fixing the problem. Here are some options to consider.

586

587 1. Drop the unit tests that involve ``example2``. This makes
588 sense, because ``example2`` does not conform to the required
589 invariants.

590

591 This is a **lost opportunity** since testing on some large
592 cases is a useful thing to do.

593
594 2. Change the parity of the offending unit tests, that is,
595 change the `member` unit test to
596
597 `unit_test (member 1322 example2) "member: 1322 in e2";`
598
599 This is a **terrible idea**. It casts in stone the particular
600 way in which the function behaves on invariant-failing cases,
601 even though, the function's specification doesn't require
602 this behavior.
603
604 3. Change the definitions of the `member`, `intersection`, and
605 `union` functions so that they verify the invariants on their
606 arguments and results. But what should they do if the
607 verification fails? They might raise an appropriate
608 exception, which the unit tests could then test for. But the
609 raising of the exception in that case was not part of the
610 specifications of those functions.
611
612 a. In addition, add a specification to the functions as to
613 what exceptions may be raised and under what
614 conditions. Then add unit tests that verify that behavior.
615
616 This is a **reasonable choice** though it requires a fair
617 amount of effort. In later labs we'll see methods for
618 making sure that nonconforming data structures can be
619 prevented from even being created.
620
621 4. Change the definition of `example2` to obey the
622 invariants. There are many ways to do so. For instance, we
623 might drop the offending 1872 from the example, or move it to
624 the end of the list where it would be in sorted order. In the
625 solution version of `labD\_examples.ml`, we replaced the 1872
626 with 1072, which falls properly in sorted order.
627 *)